



Chapter 2: Intro to Relational Model

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- **Relational Model**
 - Structure of Relational Databases
 - Keys
 - Database Schema
 - Schema Diagrams
- Relational Query Languages
 - The Relational Algebra



关系模型

- 关系模型是一种数据模型，是关系数据库的理论基础
- 数据模型的三要素
 - 数据结构
 - 数据操作
 - 数据完整性
- 关系模型的三要素
 - 单一数据结构 —— 关系
 - 数据操作 —— 关系操作
 - 数据完整性 —— 关系完整性

什么是关系？



关系的定义：域

- 相同类型值的集合
- 例子
 - $D_1: \{ 1, 2, 3 \}$.
 - $D_2: \{ \text{Female}, \text{Male} \}$
 - $D_3: \{ \text{"Jones"}, \text{"Smith"}, \text{"Jackson"} \}$
 - 反例： $\{ 1, \text{'Jones'} \}$ 不是域
- 域的基数：域中值的个数
 - 域 D_1 的基数为3



笛卡儿积

- 笛卡儿积 $D_1 \times D_2 \times \dots \times D_n$, $D_1, D_2, \dots, D_n$ 是域（可有重复）
 - 所有这样的有序组合构成了该笛卡儿积：第一个元素来自 D_1 ，第二个元素来自 D_2 ，... 第 n 个元素来自 D_n
 - **$D_1 \times D_2 \times \dots \times D_n = \{ (d_1, d_2, \dots, d_n) \mid d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n \}$**
 - 每个组合 $(d_1, d_2, \dots, d_n)$ 称为元组，其中的各个值 d_i （来自域 D_i ）称为分量。有 n 个分量的元组又称为 n -元组。
 - 笛卡儿积中元组的个数，称为**笛卡儿积的基数**



笛卡儿积

- 给定域
 - $D_1: \{ 1, 2, 3 \}$.
 - $D_2: \{ \text{Female}, \text{Male} \}$
 - $D_3: \{ \text{"Jones"}, \text{"Smith"}, \text{"Kate"} \}$
- 问题：对于笛卡儿积 $D_1 \times D_2 \times D_3$,
 - $(1, \text{'Jones'})$ 是它的一个元组么？
 - $(1, \text{'Jones'}, \text{Male})$ 是它的一个元组么？
 - $(1, \text{Male}, \text{'Jones'})$ 是它的一个元组么？
 - $D_1 \times D_2 \times D_3$ 有多少个元组？
- 结论：笛卡儿积 $D_1 \times D_2 \times \dots \times D_n$ 的基数 = 域 D_1 的基数 $\times$ 域 D_2 的基数 $\times \dots \times$ 域 D_n 的基数



笛卡儿积

1
2
3

D_1

Female
Male

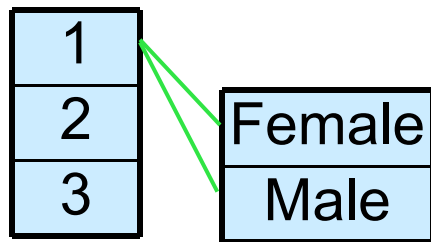
D_2

?

$D_1 \times D_2$



笛卡儿积



D_1

D_2

1	Female
1	Male

$D_1 \times D_2$



笛卡儿积

1
2
3

D_1

Female
Male

D_2

1	Female
1	Male
2	Female
2	Male

$D_1 \times D_2$



笛卡儿积

1
2
3

D_1

Female
Male

D_2

1	Female
1	Male
2	Female
2	Male
3	Female
3	Male

$D_1 \times D_2$



笛卡儿积

1
2
3

D_1

Female
Male

D_2

1	Female
1	Male
2	Female
2	Male
3	Female
3	Male

$D_1 \times D_2$

Jones
Smith
Kate

D_3

?

$D_1 \times D_2 \times D_3$



笛卡儿积

1
2
3

D_1

Female
Male

D_2

1	Female
1	Male
2	Female
2	Male
3	Female
3	Male

$D_1 \times D_2$

Jones
Smith
Kate

D_3



1	Female	Jones
1	Female	Smith
1	Female	Kate
1	Male	Jones
1	Male	Smith
1	Male	Kate
2	Female	Jones
2	Female	Smith
2	Female	Kate
2	Male	Jones
2	Male	Smith
2	Male	Kate
3	Female	Jones
3	Female	Smith
3	Female	Kate
3	Male	Jones
3	Male	Smith
3	Male	Kate

$D_1 \times D_2 \times D_3$



笛卡儿积

- 笛卡儿积的元组代表了“所有可能的组合”。
- 然而在这些“所有可能的组合”中，仅有一些元组是“有意义的，事实成立的组合”，它们反映了现实世界中的物体和属性。
- 这些元组构成了笛卡儿积的一个子集，赋予其名字，就是关系。

1	Female	Jones
1	Female	Smith
1	Female	Kate
1	Male	Jones
1	Male	Smith
1	Male	Kate
2	Female	Jones
2	Female	Smith
2	Female	Kate
2	Male	Jones
2	Male	Smith
2	Male	Kate
3	Female	Jones
3	Female	Smith
3	Female	Kate
3	Male	Jones
3	Male	Smith
3	Male	Kate

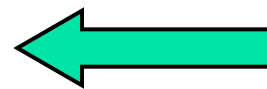


关系的定义

- 数学上的关系
 - 定义在 $D_1, D_2, \dots, D_n$ 上的关系 R 是笛卡儿积 $D_1 \times D_2 \times \dots \times D_n$ 的一个命名子集（由有意义的元组构成）
 - R 即是关系名。

student

1	Male	Jones
2	Male	Smith
3	Female	Kate



1	Female	Jones
1	Female	Smith
1	Female	Kate
1	Male	Jones
1	Male	Smith
1	Male	Kate
2	Female	Jones
2	Female	Smith
2	Female	Kate
2	Male	Jones
2	Male	Smith
2	Male	Kate
3	Female	Jones
3	Female	Smith
3	Female	Kate
3	Male	Jones
3	Male	Smith
3	Male	Kate



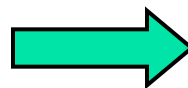
关系的定义

- 为了区分关系中的列，列又称为属性，并取唯一的名字
- 假设属性名是 $A_1, A_2, \dots, A_n$ ，那么关系 R 可以简记为：

$$R (A_1 , A_2, \dots, A_n)$$

student

student-no	sex	name
1	Male	Jones
2	Male	Smith
3	Female	Kate

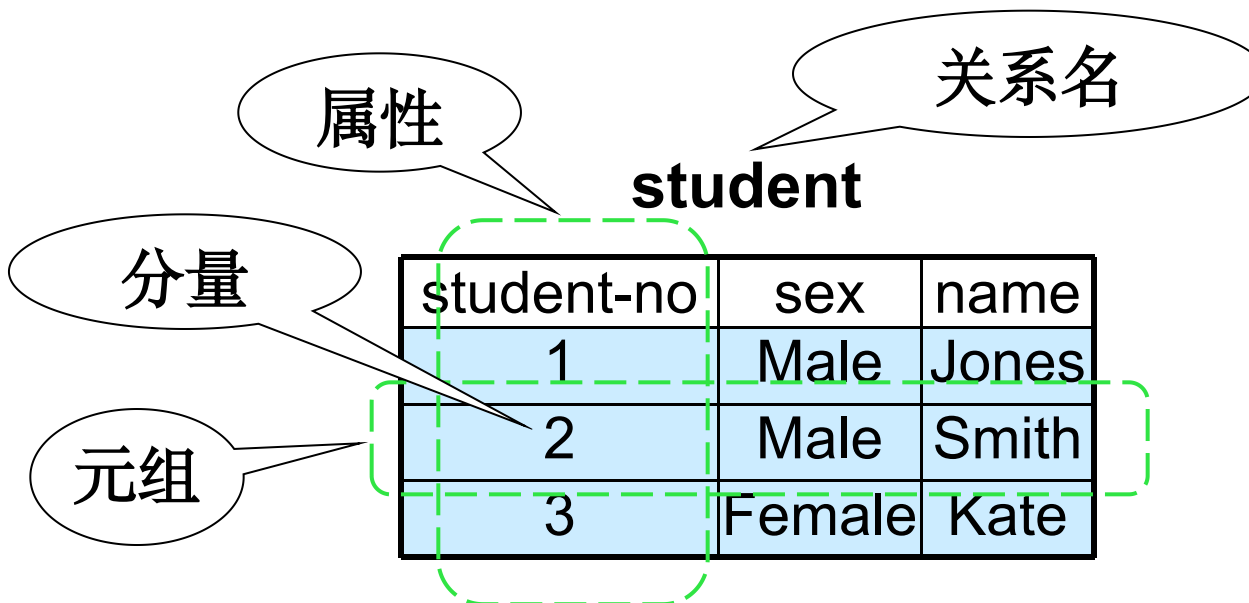


student (student-no, sex, name)



关系的组成

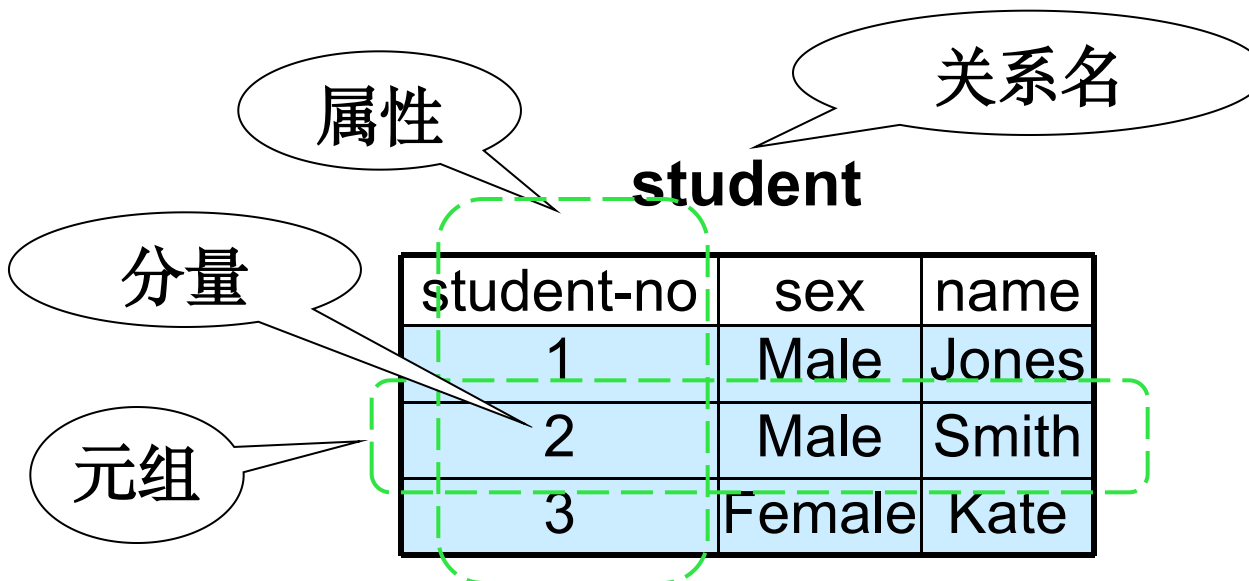
- 关系是一个命名的二维表
 - 关系的一列是属性，属性必定有名字（属性名）
 - 关系的一行是元组
 - 分量是行和列的交叉，或者说，分量是某个元组的某个属性（值）





关系的组成

- 关系的度: 属性的数目
- 关系的基数: 元组的数目
- 属性的域: 属性可能取值的集合





关系的性质（必须满足的条件）

■ 关系的性质

- 列是同质的。列中的分量类型必须相同，因为它们来自同一个域。
- 不同的列可来自同个域，但属性名不能相同
- 关系里的任意两行不能相同，因为关系是一个元组集合
 - 数学上严格禁止，但是在实际数据库里面可以容忍重复
- 关系的行或列是无序的。即改变行或列的次序（例如交换两行或者两列），关系不会变
- 属性值(分量)必须是原子的(不可分)。满足该条件的关系称作第一范式
 - 例如，多值属性，复合属性都是非原子的



以下二维表是关系么？

R1

A	B	C
1	“abc”	“c”
2	“de”	“a”
3	“f”	2

R2

A	B	A
1	“abc”	“c”
2	“de”	“a”
3	“f”	“b”

R3

A	B	C
1	“abc”	“c”
2	“de”	“a”
1	“abc”	“c”

R4

A	B	C
1, 3	“abc”	“c”
2	“de”	“a”
3	“f”	“b”



Example of a *Instructor* Relation

attributes (or columns)			
		-	

tuples
(or rows)



Data Mining: Data Sets

n Record

- | Relational records
- | Data matrix, e.g., numerical matrix, crosstabs
- | Document data: text documents: term-frequency vector
- | Transaction data

n Spatial, image and multimedia:

- | Spatial data: maps
- | Image data:
- | Video data:

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0



Relation Schema and Instance

- $A_1, A_2, \dots, A_n$ are *attributes*
- $R = (A_1, A_2, \dots, A_n)$ is a *relation schema*

Example:

instructor = (*ID*, *name*, *dept_name*, *salary*)

- A relation *instance* r defined over schema R is denoted by $r(R)$.
- The current values a relation are specified by a table
- An element t of relation r is called a *tuple* and is represented by a *row* in a table



Attributes

- The set of allowed values for each attribute is called the **domain** of the attribute
- Attribute values are (normally) required to be **atomic**; that is, indivisible
- The special value ***null*** is a member of every domain. Indicated that the value is “unknown”
- The null value causes complications in the definition of many operations



Relations are Unordered

- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- Example: *instructor* relation with unordered tuples

		-	



Keys

- Let $K \subseteq R$
- K is a **superkey** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$
 - Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of *instructor*.
- Superkey K is a **candidate key** if K is minimal
Example: $\{ID\}$ is a candidate key for *Instructor*
- One of the candidate keys is selected to be the **primary key**.
- **Foreign key** constraint: Value in one relation must appear in another
 - **Referencing** relation
 - **Referenced** relation
 - Example: *dept_name* in *instructor* is a foreign key from *instructor* referencing *department*



关系的码

■ 外部码

- 关系 R_1 中的单个属性或属性集合，对应关系 R_2 的主码
 - R_1 称为参照关系， R_2 称为被参照关系
- 例如，“student”关系中的dept-no就是一个外部码，对应“deparment”关系的主码

department

<u>dept-no</u>	dept-name
D1	计算机科学
D2	经济管理

student

dept-no	<u>student-no</u>	name	sex
D1	S1	张三	男
D1	S2	李四	男
D2	S3	王五	女



关系的码

■ 思考

- 将student的dept-no属性更名为dept， 它还是一个外部码吗？
- 换句话说，如何正确理解“外部码参照(引用)主码”？——值的参照，而不是名字的参照。
- **重要结论：** 外部码和对应主码的属性名不一定要相同。

department

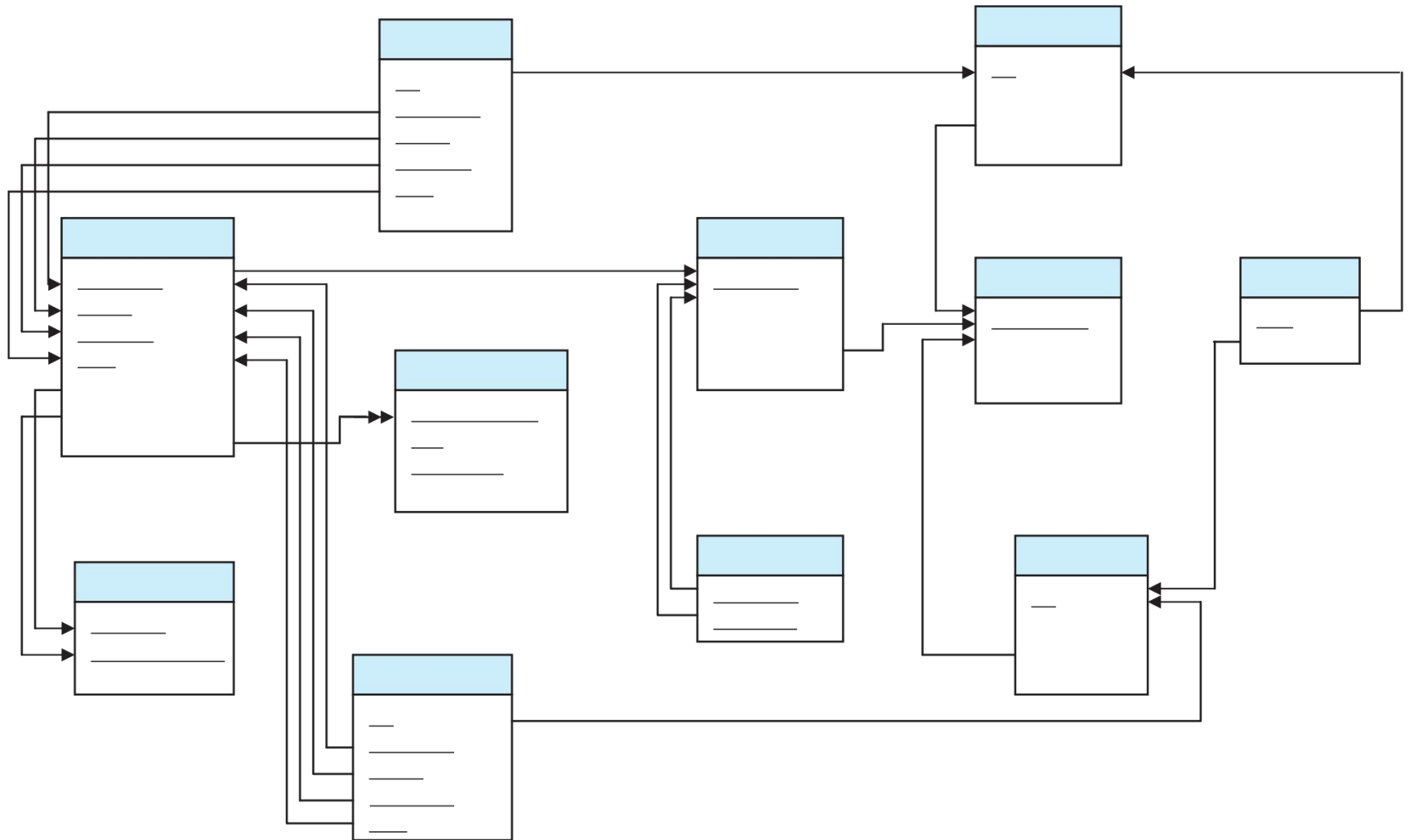
<u>dept-no</u>	dept-name
D1	计算机科学
D2	经济管理

student

dept	<u>student-no</u>	name	sex
D1	S1	张三	男
D1	S2	李四	男
D2	S3	王五	女



Schema Diagram for University Database





Outline

- Relational Model
 - Structure of Relational Databases
 - Keys
 - Database Schema
 - Schema Diagrams
- Relational Query Languages
 - The Relational Algebra



Relational Query Languages

- Procedural versus non-procedural, or declarative
- “Pure” languages:
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate in this chapter on relational algebra
 - Consists of 6 basic operations



Relational Algebra

- A procedural language consisting of a set of operations that take **one or two relations** as input and produce a new relation as their result.
- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ



Select Operation

- The **select** operation selects tuples that satisfy a given predicate. (σ : **sigma**)
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the “Physics” department.

- Query

$$\sigma_{dept_name = \text{“Physics”}}(instructor)$$

- Result

		–	



Select Operation (Cont.)

- We allow comparisons using

$=, \neq, >, \geq, <, \leq$

in the selection predicate.

- We can combine several predicates into a larger predicate by using the connectives:

$\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

- Example: Find the instructors in Physics with a salary greater \$90,000, we write:

$\sigma_{dept_name = \text{"Physics"} \wedge salary > 90,000} (instructor)$

- The select predicate may include comparisons between two attributes.
 - Example, find all departments whose name is the same as their building name:
 - $\sigma_{dept_name=building} (department)$

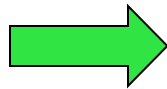


选择运算(σ) – 例子

- 关系 r

A	B	C	D
a	a	1	7
a	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \wedge D > 5}(r)$



A	B	C	D
a	a	1	7
β	β	23	10



思考

- 下列选择运算的结果是什么？

$$\sigma_{1=1}(student)$$

student

student-no	name	sex
1	Jones	Male
2	Smith	Male
3	Kate	Female



?

student-no	name	sex
1	Jones	Male
2	Smith	Male
3	Kate	Female



思考

- 下列选择运算的结果是什么？

$$\sigma_{1=2}(student)$$

student

student-no	name	sex
1	Jones	Male
2	Smith	Male
3	Kate	Female



$\emptyset$





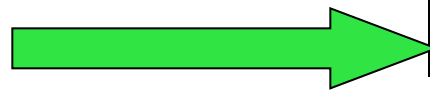
思考

- 下列选择运算的结果是什么？

$$\sigma_{1=2}(\text{student})$$

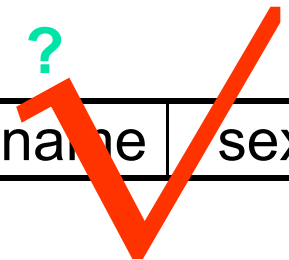
student

student-no	name	sex
1	Jones	Male
2	Smith	Male
3	Kate	Female



student-no	name	sex
------------	------	-----

?





Project Operation

- A unary operation that returns its argument relation, with certain attributes left out. (Π : pi)
- Notation:

$$\Pi_{A_1, A_2, A_3 \dots A_k} (r)$$

where $A_1, A_2, \dots, A_k$ are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets



Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*

- Query:

$\Pi_{ID, name, salary} (instructor)$

- Result:



Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.



Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- For $r \cup s$ to be valid.
 1. r, s must have the *same* **arity** (same number of attributes)
 2. The attribute domains must be **compatible** (example: 2nd column of r deals with the same type of values as does the 2nd column of s)
- Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\Pi_{course_id} (\sigma_{semester = "Fall" \wedge year = 2017}(section)) \cup \Pi_{course_id} (\sigma_{semester = "Spring" \wedge year = 2018}(section))$$



Union Operation (Cont.)

- Result of:

$$\Pi_{course_id} (\sigma_{semester = \text{“Fall”} \wedge year = 2017} (section)) \cup \\ \Pi_{course_id} (\sigma_{semester = \text{“Spring”} \wedge year = 2018} (section))$$

—



Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
 - r, s have the *same arity*
 - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\pi_{course_id} (\sigma_{semester = \text{"Fall"} \wedge year = 2017} (section)) \cap \pi_{course_id} (\sigma_{semester = \text{"Spring"} \wedge year = 2018} (section))$$

- Result

–



Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
 - r and s must have the **same** arity
 - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{course_id} (\sigma_{semester = "Fall" \wedge year = 2017}(section)) - \Pi_{course_id} (\sigma_{semester = "Spring" \wedge year = 2018}(section))$$





并,交,差运算 – 例子

- 关系 r, s :

r

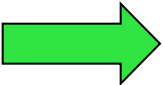
A	B
a	1
a	2
β	1

s

A	B
a	2
β	3

$r \cup s$:

A	B
a	1
a	2
a	2
β	1
β	3



A	B
a	1
a	2
β	1
β	3

$r \cap s$:

A	B
a	2

$r - s$:

A	B
a	1
β	1



Cartesian-Product Operation

- The Cartesian-product operation (denoted by $\times$) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and *teaches* is written as:

instructor $\times$ *teaches*

- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
 - *instructor.ID*
 - *teaches.ID*



笛卡儿积运算(x) – 例子

关系 r, s :

r		s		
A	B	C	D	E
a	1	a	10	a
β	2	β	10	a
		β	20	b
		γ	10	b

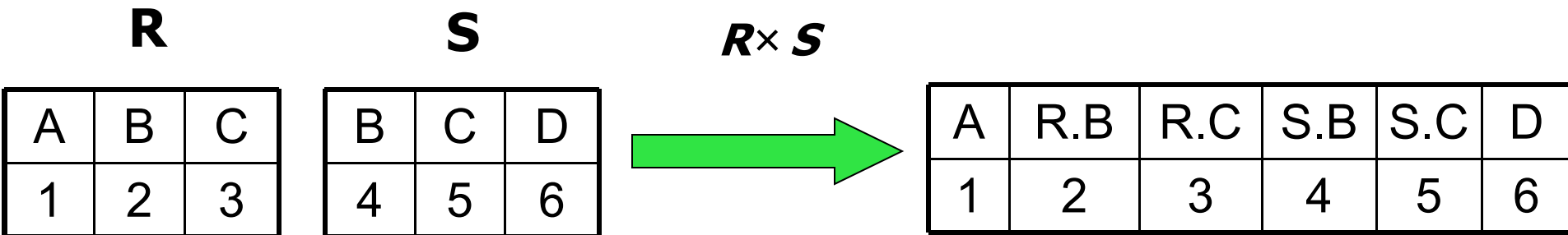
$r \times s$:

A	B	C	D	E
a	1	a	10	a
a	1	β	10	a
a	1	β	20	b
a	1	γ	10	b
β	2	a	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



笛卡儿积运算(x)

- 笛卡儿积运算的例2





Join Operation

- The Cartesian-Product

instructor X teaches

associates every tuple of *instructor* with every tuple of *teaches*.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught, we write:

$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$

- We get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide



运算的复合: 关系代数表达式

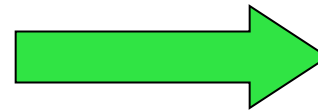
- 一个关系运算+输入关系，本身就是一个简单的关系代数表达式
 - 例如， $r \times s$, $\sigma_{1=1}(r)$
- 可以用多个运算的复合来构建复杂的关系代数表达式
 - 例如， $\sigma_{A=C}(r \times s)$

r

<i>A</i>	<i>B</i>
<i>a</i>	1
β	2

s

<i>C</i>	<i>D</i>	<i>E</i>
<i>a</i>	10	<i>a</i>
β	10	<i>a</i>
β	20	<i>b</i>
γ	10	<i>b</i>



r x s

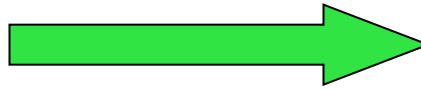
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>a</i>	1	<i>a</i>	10	<i>a</i>
<i>a</i>	1	β	10	<i>a</i>
<i>a</i>	1	β	20	<i>b</i>
<i>a</i>	1	γ	10	<i>b</i>
β	2	<i>a</i>	10	<i>a</i>
β	2	β	10	<i>a</i>
β	2	β	20	<i>b</i>
β	2	γ	10	<i>b</i>



运算的复合: 关系代数表达式

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>a</i>	1	<i>a</i>	10	<i>a</i>
<i>a</i>	1	β	10	<i>a</i>
<i>a</i>	1	β	20	<i>b</i>
<i>a</i>	1	γ	10	<i>b</i>
β	2	<i>a</i>	10	<i>a</i>
β	2	β	10	<i>a</i>
β	2	β	20	<i>b</i>
β	2	γ	10	<i>b</i>

$\sigma_{A=C}(r \times s)$



<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
<i>a</i>	1	<i>a</i>	10	<i>a</i>
β	2	β	20	<i>a</i>
β	2	β	20	<i>b</i>



Join Operation (Cont.)

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations $r (R)$ and $s (S)$
- Let “theta” be a predicate on attributes in the schema $R \cup S$. The join operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

- Thus

$$\sigma_{\text{instructor.id} = \text{teaches.id}} (\text{instructor} \times \text{teaches})$$

- Can equivalently be written as

$$\text{instructor} \bowtie_{\text{instructor.id} = \text{teaches.id}} \text{teaches.}$$



The Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ , is provided for that purpose
- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A1,A2, \dots, An)}(E)$$

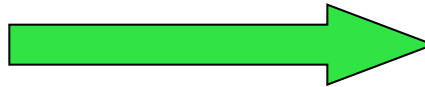


重命名运算(ρ)

student

<u>student-no</u>	name	sex
1	Jones	Male
2	Smith	Male
3	Kate	Female

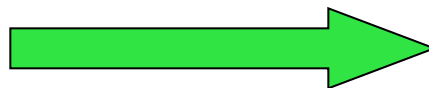
$\sigma_{sex='Male'}(student)$



?

<u>student-no</u>	name	sex
1	Jones	Male
2	Smith	Male

$\rho_{boy}(\sigma_{sex='male'}(student))$



boy

<u>student-no</u>	name	sex
1	Jones	Male
2	Smith	Male



重命名运算(ρ)

- 格式2:

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

- 下标表示将E的结果关系命名为x, 同时属性依次命名为 $A_1, A_2, \dots, A_n$



重命名运算(ρ)

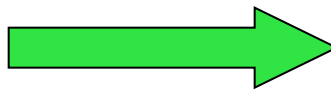
R

A	B	C
1	3	3

S

B	C	D
4	5	6

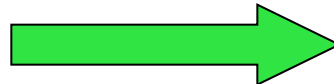
$R \times S$



A	R.B	R.C	S.B	S.C	D
1	2	3	4	5	6

$\rho_{V(RA, RB, RC, SB, SC, SD)} (R \times S)$

V



RA	RB	RC	SB	SC	RD
1	2	3	4	5	6



The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

$Physics \leftarrow \sigma_{dept_name = \text{“Physics”}}(instructor)$

$Music \leftarrow \sigma_{dept_name = \text{“Music”}}(instructor)$

$Physics \cup Music$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.



Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department with salary greater than 90,000

- Query 1

$$\sigma_{dept_name = \text{"Physics"} \wedge salary > 90,000} (instructor)$$

- Query 2

$$\sigma_{dept_name = \text{"Physics"}} (\sigma_{salary > 90,000} (instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.



Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department
- Query 1

$\sigma_{dept_name = \text{"Physics"}} (instructor \bowtie_{instructor.ID = teaches.ID} teaches)$

- Query 2

$(\sigma_{dept_name = \text{"Physics"}} (instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.



End of Chapter 2